Week 11 – Solution Lab: AI Line Following

Reference solutions for all Week 11 labs. Read the theory first, attempt independently, then compare with these solutions.

AI ROBOTICS

OPENCV · CONTROL THEORY · SIMULATION

Lab Overview

01

Lab 11.1

Offline image processing — compute line error from video

02

Labs 11.2–11.3

Simulate unicycle & Ackermann robots with P controller

03

Lab 11.4

Combine vision pipeline + controller in real-time simulation

04

Labs 11.5–11.7

Deploy on Jet Racer (PD) and Turtlebot hardware

Lab 11.1 – Offline Image Processing

Goal: Read a video/image sequence of a track, compute error *e(t)* over time, and plot it to visualize track difficulty.

ROI

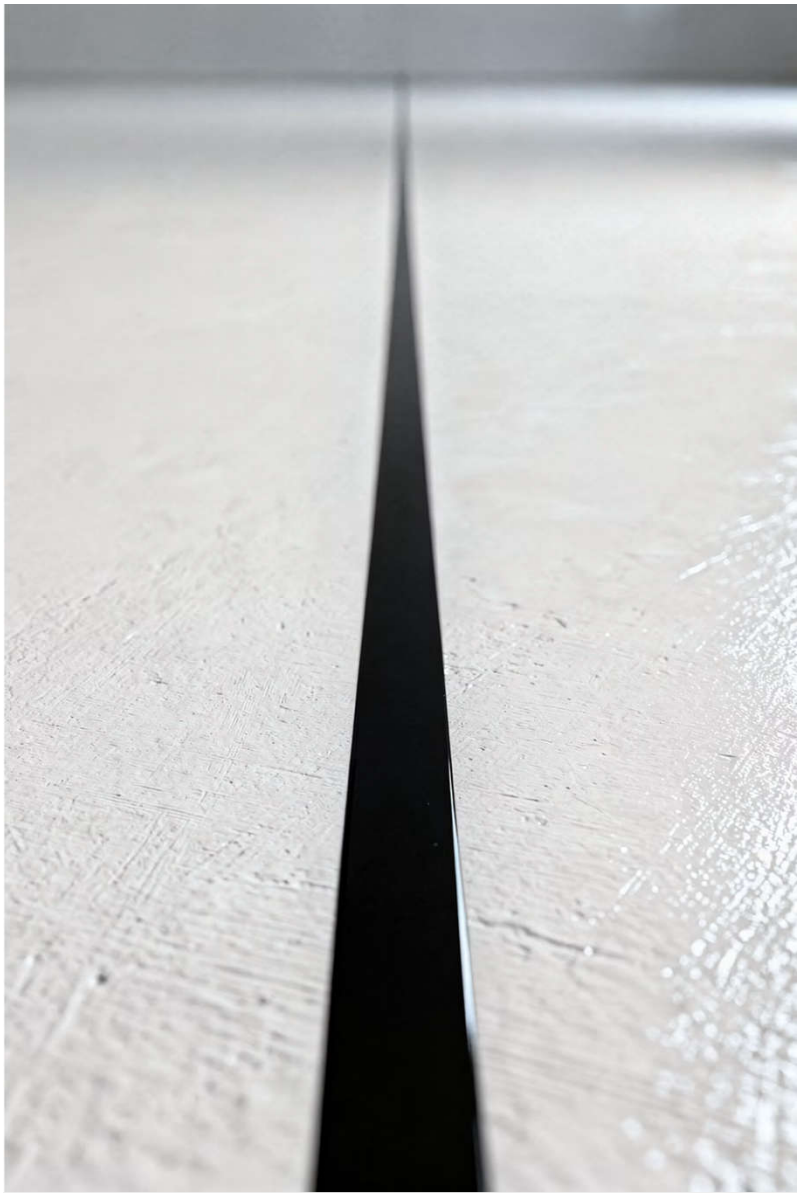
Bottom 40% of frame — focuses on the line ahead

Otsu Threshold

Auto-selects threshold — robust under changing light

Centroid

Finds the "center" of the line in the ROI



Lab 11.1 – Core Function

line_error_from_frame()

```
gray = cv2.cvtColor(frame, BGR2GRAY)
roi = gray[int(h*0.6):h, :]
blurred = cv2.GaussianBlur(roi, (5,5), 0)
_, bw = cv2.threshold(blurred, 0, 255,
    THRESH_BINARY_INV + THRESH_OTSU)
M = cv2.moments(bw)
cx = int(M['m10'] / M['m00'])
e = (cx - w/2.0) / (w/2.0)  # ∈ [-1, 1]
```

Error Convention

- $e < 0$ → line shifted left
- $e > 0$ → line shifted right
- $e = 0$ → line centered

Output: plot of e(t) saved as `lab11_1_error_plot.png`. Check `max(|e|)` to assess track difficulty.

Lab 11.2 – Unicycle Simulator + P Controller

Goal: Simulate a unicycle robot tracking a sine-wave reference using a P controller on lateral error.

unicycle_step()

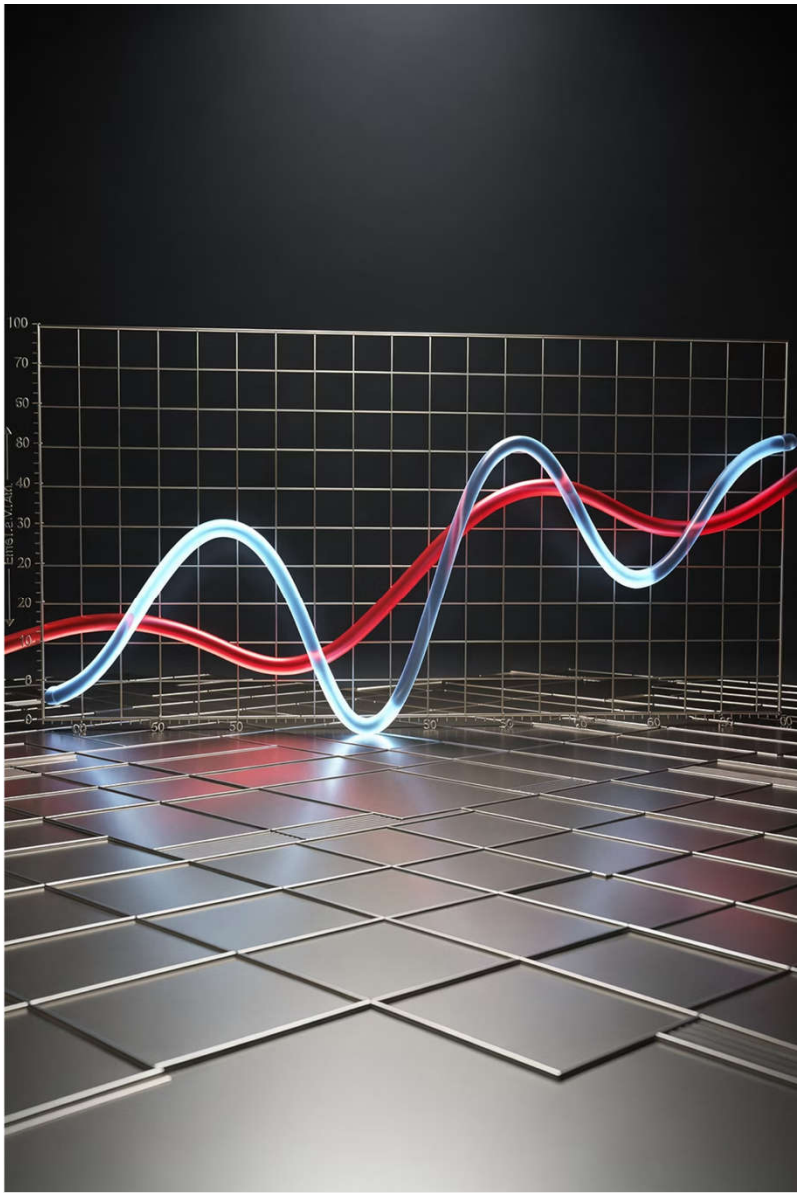
Updates (x, y, θ) using velocity v and angular rate ω

lateral_error()

Signed perpendicular distance from robot to reference point

Lookahead

0.3 m ahead on the reference — avoids control lag



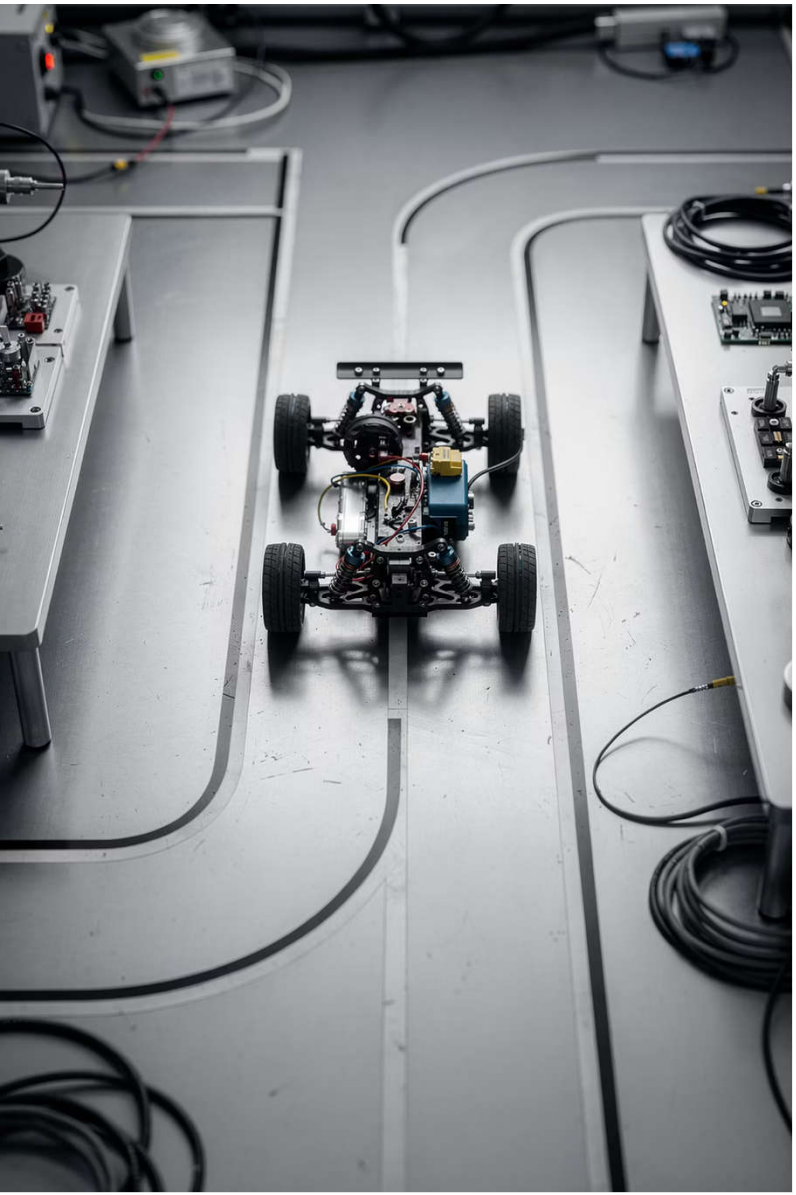
Lab 11.2 – Key Parameters & Logic

```
Kp = 1.5      # P gain
v_const = 0.5 # forward speed (m/s)
dt = 0.05     # timestep (s)
T  = 20.0     # total time (s)

# Per step:
x_ref, y_ref = reference_line(t + 0.3)
e = lateral_error(x, y, th, x_ref, y_ref)
w = Kp * e
x, y, th = unicycle_step(x, y, th,
                         v_const, w, dt)
```

Tuning Tip

Increase **Kp** to track faster curves — but watch for overshoot and oscillation.

lateral_error uses a cross-product projection: `e = -sin(θ)·dx + cos(θ)·dy`



Lab 11.3 – Ackermann (Jet Racer) Simulator

Goal: Replace unicycle kinematics with Ackermann steering to simulate the Jet Racer platform.

ackermann_step()

Updates pose using steering angle δ and wheelbase $L = 0.25\ m$

delta_max

Mechanical limit: ±25° — steering is clipped to this range

Convergence

Ackermann can't spin in place — large errors take longer to correct

Lab 11.3 – Ackermann Core Logic

```
def ackermann_step(x, y, th, v, delta, L, dt):
    x  += v * np.cos(th) * dt
    y  += v * np.sin(th) * dt
    th += (v / L) * np.tan(delta) * dt
    return x, y, th

# Steering command:
delta = np.clip(Kp * e, -delta_max, delta_max)
```

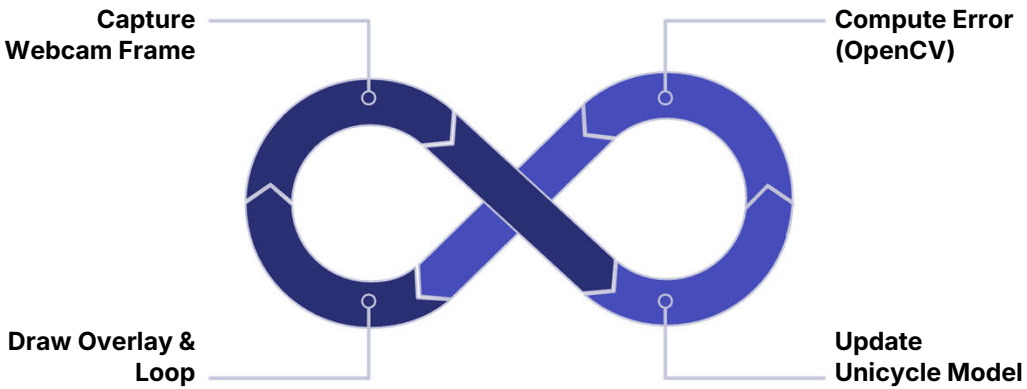
vs. Unicycle

- Unicycle: direct ω command
- Ackermann: steering angle δ, limited by geometry
- Same lateral_error function reused from Lab 11.2

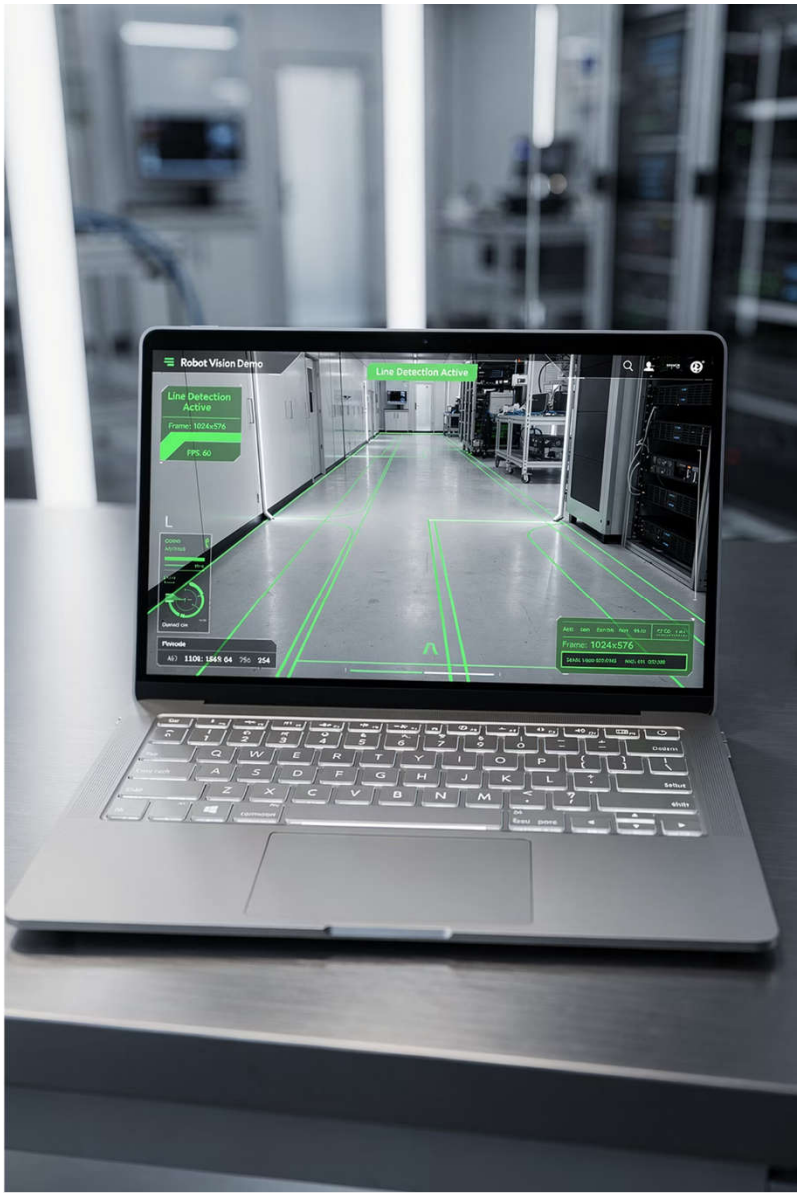
Kp = 1.2 works well for Jet Racer at 0.5 m/s on a gentle sine track.

Lab 11.4 – Vision Pipeline + Real-Time Controller

Goal: Connect the OpenCV image pipeline to the unicycle controller in a live webcam loop — validate before deploying on hardware.



The loop runs at ~30 fps. If the line is lost, error defaults to 0 (robot drives straight). Overlay shows `e` and `ω` values live on the frame.



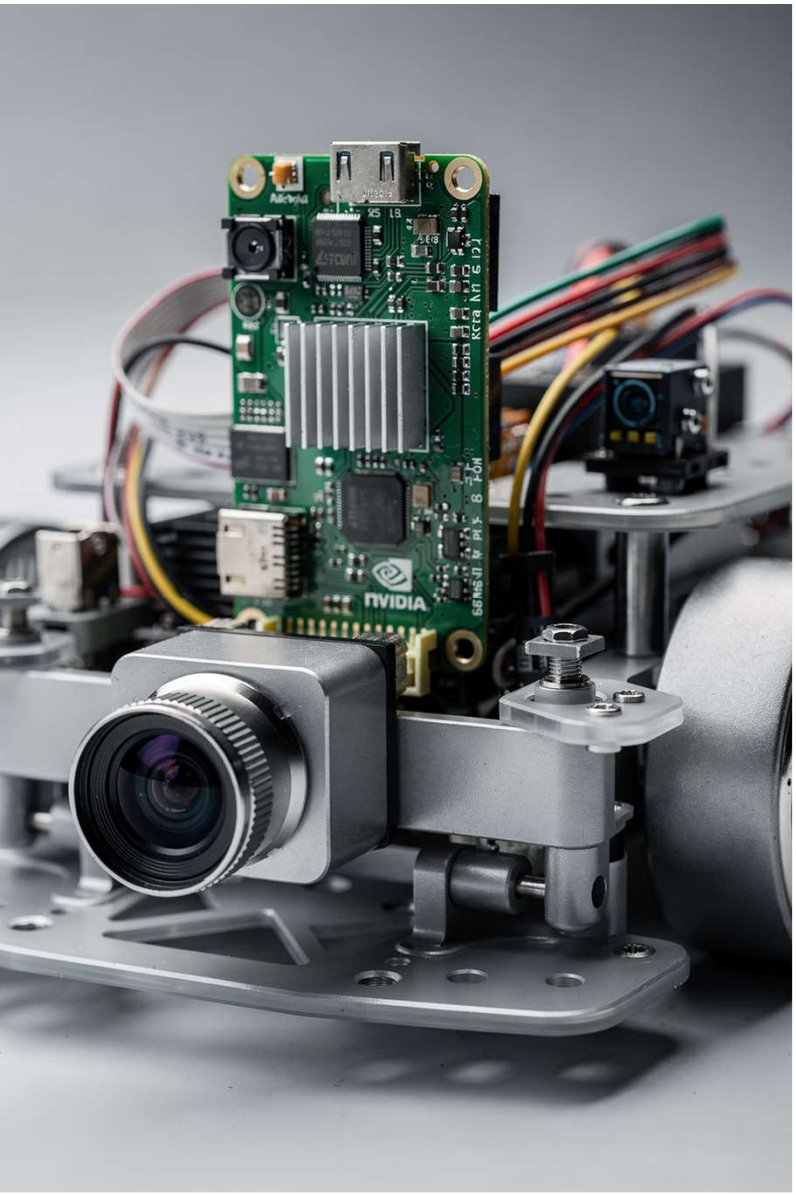
Lab 11.4 – Overlay & Trajectory

```
e, bw = line_error_from_frame(frame)
if e is None: e = 0.0  # keep straight
w = Kp * e
rx, ry, rth = unicycle_step(
    rx, ry, rth, v_const, w, dt)

# Draw centroid dot (red):
cx = int((e * ww/2) + ww/2)
cv2.circle(frame, (cx, int(h*0.8)),
           8, (0,0,255), -1)
cv2.putText(frame,
    f'e={e:+.3f} w={w:+.3f}', ...)
```

What to Check

- Red dot tracks the line centroid correctly
- Simulated trajectory is smooth, not erratic
- Error stays near 0 on straight sections

If pipeline works here, it's ready for the real car.



Lab 11.5 – Jet Racer Road Following (NVIDIA/Waveshare)

Uses the **jetracer SDK** with a ResNet-18 regression model trained to predict a target point *(x, y)* on the road image.

1

Collect Data

Click target points in `road_following_data_collection.ipynb`

2

Train Model

ResNet-18 regression, ~10–20 min on Jetson Nano

3

Run Live

Load model, run inference loop, map x_target → steering

Lab 11.5 – Inference Loop

```
with torch.no_grad():
    xy = model(preprocess(frame)).squeeze()
    x_t = xy[0].item()  # ∈ [-1, 1]

car.steering = float(np.clip(x_t, -1, 1))
car.throttle = 0.18
```

Preprocessing: BGR→RGB, normalize with ImageNet mean/std, send to CUDA.

Key Insight

Mapping `x_target → steering` directly is an **implicit P controller** with Kp = 1.

> Model output: normalized (x, y) coordinates of the desired road point. Only x is used for steering.

Lab 11.6 – Replacing P with PD Controller

Upgrade the Jet Racer steering from simple proportional to **PD control** to reduce overshoot and oscillation.

```
Kp = 0.8;  Kd = 0.15
prev_error = 0.0

# In inference loop:
e  = x_t          # error vs. center (0.0)
de = e - prev_error
steer = Kp*e + Kd*de
steer = float(np.clip(steer, -1.0, 1.0))
car.steering = steer
car.throttle = 0.18
prev_error = e
```

What the D Term Does

Kd·de dampens the steering when error is *decreasing fast* — prevents over-correction.

- Typical Kd range: **0.1–0.2** at low speed
- Higher throttle → lower Kp or higher Kd

Lab 11.6 – Gain Tuning Workflow

1

Set Kd = 0

Tune Kp until the car tracks but still oscillates slightly

2

Increase Kd

Raise gradually to reduce oscillation without losing tracking

3

Increase Throttle

Raise speed slightly, then re-check and re-tune gains

Kp too high

Zigzag / strong oscillation

Kp too low

Slow tracking, exits line on curves

Kd too high

Jerky / sudden steering jolts

Lab 11.7 – Turtlebot Line Following

Uses the **iRobot Education Python SDK** (or compatible driver). No ROS required — commands sent directly via Bluetooth/serial.

Vision

Same `line_error_from_frame()` pipeline as Lab 11.1

PD Control

Kp = 1.2, Kd = 0.1 → compute angular velocity ω

diff_drive_ik()

Converts (v, ω) → (v_L, v_R) wheel speeds in mm/s

Lab 11.7 – Differential Drive IK & Safety

```
def diff_drive_ik(v, w, W=0.235):
    """W = Turtlebot 3 track width (m)"""
    v_R = v + w * W/2
    v_L = v - w * W/2
    return v_L, v_R

# If line lost → stop:
if e is None:
    robot.set_wheel_speeds(0, 0)
    continue

# Send command (m/s → mm/s):
robot.set_wheel_speeds(
    int(v_L*1000), int(v_R*1000))
```

Safety: finally Block

Always wrap the main loop in `try/finally` to guarantee the robot stops when the script exits.

> ⚠ Skipping the finally block risks the robot continuing to move after the program crashes.

Extension – Adaptive Throttle

Automatically reduce speed when lateral error is large — giving the robot more time to react before leaving the track.

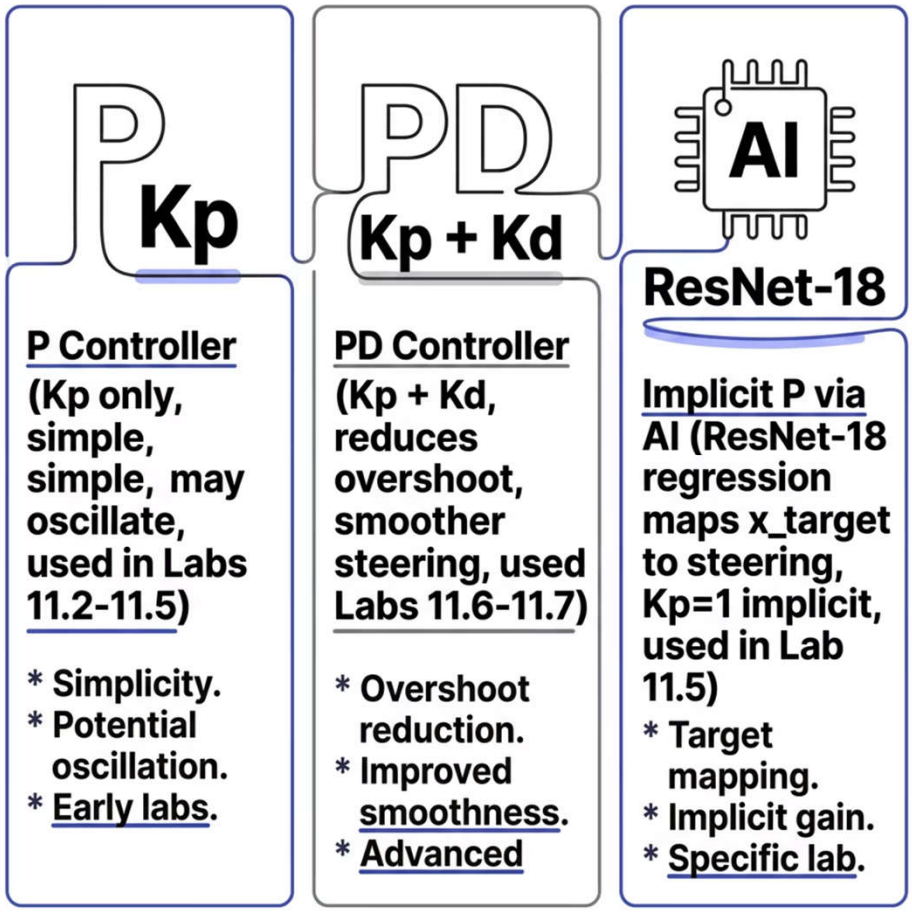
```
def adaptive_throttle(e,
        base_throttle=0.20,
        min_throttle=0.10):
    t = base_throttle * (1.0 - abs(e))
    return max(t, min_throttle)

# In loop:
throttle = adaptive_throttle(e)
car.throttle = throttle
```

Why It Works

When $|e| \to 1$ (large deviation), throttle drops toward `min_throttle`. When $|e| \to 0$ (on track), full speed is restored.

Simple but highly effective in practice — especially on tight curves.

Controller Comparison



Choosing the Right Controller

- **P only**: Good starting point, easy to tune
- **PD**: Better for higher speeds and tighter tracks
- **AI regression**: Handles complex road appearance, no manual error formula needed

All three share the same core idea: measure deviation, correct proportionally.

Key Concepts Summary

Vision Pipeline

ROI crop → Gaussian blur → Otsu threshold → centroid → normalized error $e \in [-1, 1]$

Kinematics

Unicycle: direct ω. Ackermann: steering angle δ with mechanical clip. Diff-drive: IK to wheel speeds.

Control

P controller: fast to implement. PD: smoother. Adaptive throttle: safety at large errors.

AI on Hardware

ResNet-18 regression on Jetson Nano predicts road target; maps directly to steering command.

Week 11 Complete

You've implemented line following from raw pixels to real hardware — across three robot platforms and three controller types.

Next Step

Tune PD gains on your platform, then try adaptive throttle for smoother laps

Challenge

Add a PID (integral term) to eliminate steady-state offset on banked or asymmetric tracks